

ADVANCED COMPUTER ARCHITECTURE

Examples of problems

1. Modify the attached multi-cycle processing unit to implement the instruction ``is3mult $s0, $s1, $s2``, which checks whether the digits contained in the vector pointed to by register `$s1` and of the size specified in register `$s2` are multiples of 3, returning the result to register `$s0`. If, instead, you were to check for multiples of 4, propose an optimized version of the same procedure. Justify your choices and explain them as clearly as possible.
2. Modify the attached multi-cycle processor to implement the instruction ``lxi $s0, offset ($s1)``, which loads bytes, halfwords, or words from memory while automatically incrementing the pointer to the next memory location.
3. Modify the attached multi-cycle processing unit to implement the ``even_parity_check $s0, $s1`` instruction, which counts the number of bits set to "1" within a message frame contained in the array starting at the address in `$s1`, after discarding the first 12 elements. The vector ends with a byte equal to NULL. The preceding bit is the so-called parity bit. If this bit matches the parity of the count, the sequence is correct; otherwise, an exception occurs with a jump to the address contained in `$s0`.
4. Modify the attached multi-cycle processing unit to implement the instruction ``call_lessthan $s0, $s1, $s2``, which calls a routine located at the address contained in register `$s0` if the value of register `$s1` is less than that of register `$s2`.

Indicate what modification the previous instruction would introduce in control units implemented as:

- 1) future-state sequencing units;
 - 2) a pair of ROMs containing the present-future state and the present-controls state
5. Modify the attached multi-cycle processor to implement the instruction ``fact $s0, $s1``, which calculates the factorial of the number in register `$s0` and returns the result to register `$s1`.
 6. Implement the following instruction: *New_Tart* (`$s0, $s1, $s2`)

which implements the expansion of a binomial $(a+b)^n$ using the first N rows of Tartaglia's triangle, where n is stored in `$s0` and a and b are stored in the two registers (`$s1` and `$s2`).

For example, if $n=3$, considering the correct row of the triangle, the calculation can be implemented as follows:

$$(a+b)^3 = a^3 + 3a^2b + 3ab^2 + b^3 = 1 \cdot a^3 + 3 \cdot a^2b + 3 \cdot ab^2 + 1 \cdot b^3$$

Draw the flowchart of the algorithm implemented for a general n, and then the finite-state machine specifically for the case $n=3$.



